Spaces Mockup MCP Lab Guide

Easy to Advanced · Nov 12, 2026 · @Jason Churchill

Contents

- Overview and Before you start
- Data sources: the tools, locations, rooms and reference data
- Labs: 1 Rooms and environment · 2 Utilization and occupancy · 3 Devices, errors and lifecycle · 4 Configuration and drift · 5 Locations and the Firehose replay
- Stretch goals
- Appendix: client setup
- Answer key

Overview

This guide covers all 19 tools on the `spaces-mockup` MCP server and nothing else. The server serves a read-only sample tenant, Mockup Campus in Sydney, so nothing you ask can change real systems. There are five labs, each with an Easy track and an Advanced track, and stretch goals at the end. Allow about 10 minutes for setup, then 50 minutes for the Easy track only, or about 2 hours for both tracks, plus time for any stretch goals.

You need: Claude Code, Claude Desktop, Cursor, Codex CLI, or the Codex desktop app, installed and signed in before the lab starts, and the bearer token from the lab's Box folder. Claude Desktop users also need the `spaces-mockup.mcpb` extension from the same folder. Before you start covers connecting to the `spaces-mockup` server. The labs don't use any other connectors.

How the tracks work

- **Easy:** one question, usually one tool. You learn what each data source returns.
- **Advanced:** several tools chained together, joins on IDs, or reasoning across time. You learn to combine data sources to explain something.
- **Stretch goals:** open-ended challenges with success criteria instead of a single expected answer.

Checking your answers: every exercise starts with its ID, for example **1.E1** for Lab 1, Easy track, exercise 1. The ID links to its expected answer in the Answer key at the back, so try the exercise first, and the answer's ID links back. Tick the box when you're done. If your answer differs, ask *"Which tool did you call, and with what arguments?"* Metric timestamps move with the clock, so read times relative to now, for example "about an hour ago".

Before you start

You'll connect your client to the hosted lab server at `https://spaces-mockup.mcp.buildermate.ai/mcp`. You don't need to run the server yourself.

1. **Get your token.** It's in the `.txt` file in the lab's Box folder. The steps below use `Found-In-Box-Folder` as a placeholder; replace it with your token. Do not paste the token into git, tickets, or chat.
2. **Check you can reach the server.** The health endpoint doesn't need the token. Open `https://spaces-mockup.mcp.buildermate.ai/health` in your browser, or run:
   - **Linux / macOS:**

     ```
     curl -sS https://spaces-mockup.mcp.buildermate.ai/health
     ```

   - **Windows (PowerShell):**

     ```
     curl.exe -sS https://spaces-mockup.mcp.buildermate.ai/health
     ```

   You should see:

   ```
   {"status": "ok", "server": "spaces-mockup"}
   ```

3. **Connect your client.** Follow the steps for the client you use, in the appendix:

Client	Setup steps
Claude Code on Linux or macOS	Claude Code CLI (Linux / macOS)
Claude Code on Windows	Claude Code CLI (Windows)
Claude Desktop	Claude Desktop (Windows / macOS)
Cursor	Cursor
Codex CLI	Codex CLI
Codex desktop app	Codex desktop app

Confirm it works

Whichever client you use, send this prompt before you start Lab 1:

> *"Call firehose_health."*

If your client asks permission to run the tool, allow it. The reply should say the status is ok, name the tenant as Mockup Campus, and list 14 subscribed event types.

If the tool call fails or the assistant answers without calling a tool, go back to the **Verify** step for your client in the appendix.

Start a new chat for each lab. In Claude Code, run `/clear`, or exit and relaunch `claude`. In Codex CLI, run `/new`. In the Codex desktop app, start a new chat. Every tool reply stays in the chat, and Lab 5's Firehose pulls are large, so a long chat gets slow and less reliable. Each lab stands on its own. Note down your Lab 1 model, because S4 uses it.

Data sources

You don't call the tools directly. Ask a question and the assistant (Claude, Cursor's Agent, or Codex) picks the tool.

Tools

Sites and rooms

Tool	What it returns	Input
`location_search`	Campus → building → floor tree, with address, floor count and floor number	optional filter
`workspace_search`	Rooms: type, capacity, floor, devices, capacity note	optional filter
`workspace_metrics`	Hourly series for one room: people, presence, booked, temperature °C, humidity %, noise, used and booked minutes	`workspace_id`

Utilization

Tool	What it returns	Input
`workspace_aggregated_usage_metrics`	Used and booked minutes and peak headcount per room, plus campus totals	—
`workspace_aggregated_popularity_metrics`	Rooms ranked by minutes used	—
`workspace_aggregated_capacity_utilization`	Overcrowded and under-utilized rooms with peak ratio	—

Live Firehose

Tool	What it returns	Input
`firehose_latest`	Current headcount per room and positions of tracked phones	—
`firehose_events`	Raw `EventRecord` stream, oldest first, up to 50 per pull; `truncated` and `nextOffset` say when there's more	optional `eventType`, `locationId`, `deviceId`, `workspaceId`, `fromTimestamp`, `limit`, `offset`
`firehose_health`	Stream status, tenant, subscribed event types	—

Devices

Tool	What it returns	Input
`device_search`	Model, serial, RoomOS version, room, connection status	optional filter
`device_event_history`	Recent events for one device	`device_id`
`device_aggregated_event_history`	Event counts by severity and by code across the organization	—
`device_error_code_detail_search`	Error code meaning, severity and recommended action	optional code
`product_lifecycle`	End-of-sale, software-maintenance and last-support dates per model	—

Configuration

Tool	What it returns	Input
`device_configuration_schema`	Setting keys, types, allowed values, supported models	—
`device_configuration_template_list`	Templates and the keys each one controls	—
`device_configuration_list`	Current settings on one device, and its template	`device_id`
`device_configuration_diff`	Settings that differ between two devices	`device_id_a`, `device_id_b`
`device_configuration_template_diff`	Settings where a device differs from a template	`device_id`, `template_id`

To join data across tools, use the shared IDs: `locationId`, workspace `id` / `workspaceId`, and `deviceId`.

Locations

- Mockup Campus (`loc-campus-mockup`), 1 Mockup Lane, Pyrmont NSW 2009
  - North Building (`loc-bldg-north`): North L1 (`loc-floor-n1`) and North L2 (`loc-floor-n2`)
  - South Building (`loc-bldg-south`): South L1 (`loc-floor-s1`)

Rooms and devices

Room	Room ID	Floor	Seats	Device	Device ID
Boardroom North	`ws-board`	North L1	10	Room Bar Pro	`dev-roombar-board`
Huddle A	`ws-huddle`	North L1	4	Room Kit Mini	`dev-kitmini-huddle`
Focus Desk 12	`ws-focus`	North L2	1	Desk Pro	`dev-deskpro-focus`
Phone Booth N2	`ws-phone`	North L2	1	none	—
Lab Bench	`ws-lab`	South L1	6	Board Pro 55	`dev-boardpro-lab`
Workshop Bay	`ws-workshop`	South L1	8	Room Bar	`dev-roombar-workshop`

Each device's RoomOS version and connection status are yours to find in Lab 3.

Reference data

- **Templates:** Meeting Room Standard (`tpl-meeting-standard`) and Personal Desk Standard (`tpl-desk-standard`).
- **Config keys:** `Audio.DefaultVolume`, `Video.Selfview.Default`, `Bookings.Enabled`, `Standby.Control`, `Proximity.Mode`.
- **Error codes:** `CAM_HDMI_SIGNAL_LOST` (error), `SIGNAGE_CONTENT_STALE` (warning), `PROXIMITY_PAIRING_FAIL` (warning).
- **Firehose event types (14):** `SPACE_OCCUPANCY`, `SPACE_OCCUPANCY_CHANGE`, `DEVICE_PRESENCE`, `USER_PRESENCE`, `DEVICE_LOCATION_UPDATE`, `DEVICE_ASSOCIATION`, `DEVICE_COUNT`, `CAMERA_COUNT`, `RAW_CAMERA_COUNT`, `WEBEX_TELEMETRY`, `IOT_TELEMETRY`, `PROFILE_UPDATE`, `KEEP_ALIVE`, `APP_ACTIVATION`.

Lab 1 — Rooms and environment

Goal: You'll learn to read a room's environment over time, and to compare two sources for the same reading.

Time: Easy about 10 min · Advanced about 10 min.

Tools: `workspace_search`, `workspace_metrics`, `firehose_events`.

Easy

- [ ] 1.E1 **Temperature everywhere.** *"Show me the temperature in every room right now."*
- [ ] 1.E2 **One room over the day.** *"Show Boardroom North's temperature history."*
- [ ] 1.E3 **Quietest rooms.** *"Which rooms are quietest?"*

Advanced

- [ ] 1.A1 **Comfort alerts.** *"Flag any hour in any room where temperature went over 24 °C or humidity reached 60%."*
- [ ] 1.A2 **Build a model.** *"Across all rooms, how do temperature, humidity and noise change with headcount?"*
    - Discuss: the same rule fits a 1-seat booth and a 10-seat boardroom. Is that realistic, and what does it mean for your S4 prediction?
- [ ] 1.A3 **Two sources, one room.** *"Compare Boardroom North's temperature in workspace_metrics with the WEBEX_TELEMETRY ambientTemp for the same hour."*
    - Discuss: which source would you trust for a comfort complaint, and why?
- [ ] 1.A4 **Data freshness.** *"How many readings does each room have, and how old is each room's latest one?"*

Lab 2 — Utilization and occupancy

Goal: You'll learn to tell use from bookings, and live data from history.

Time: Easy about 10 min · Advanced about 15 min.

Tools: `firehose_latest`, `workspace_aggregated_usage_metrics`, `workspace_aggregated_popularity_metrics`, `workspace_aggregated_capacity_utilization`, `workspace_metrics`.

Easy

- [ ] 2.E1 **Who's in now.** *"How many people are in each room right now?"*
- [ ] 2.E2 **Most popular.** *"Rank rooms by actual use."*
- [ ] 2.E3 **Wrong-sized rooms.** *"Which rooms are overcrowded or under-utilized?"*

Advanced

- [ ] **2.A1** **Booking accuracy.** *"For each room, divide used minutes by booked minutes and explain what the ratio means."*
- [ ] **2.A2** **No-shows and walk-ins, hour by hour.** *"Using workspace_metrics, find hours that were booked but empty, and hours that were used without a booking."*
- [ ] **2.A3** **Live vs. history.** *"Does firehose_latest agree with the latest workspace_metrics point for each room?"*
- [ ] **2.A4** **Make the case.** *"Recommend one change to the room mix. Cite the numbers from at least two tools."*
- [ ] **2.A5** **Trust the snapshot.** *"How old is each room's entry in firehose_latest? Would you use it to tell someone a room is free right now?"*

Lab 3 — Devices, errors and lifecycle

Goal: You'll learn to turn device events and lifecycle dates into a prioritized fix list.

Time: Easy about 10 min · Advanced about 10 min.

Tools: `device_search`, `device_event_history`, `device_aggregated_event_history`, `device_error_code_detail_search`, `product_lifecycle`, and `workspace_aggregated_popularity_metrics` for 3.A3.

Easy

- [ ] **3.E1** **Fleet status.** *"List all devices with status and RoomOS version."*
- [ ] **3.E2** **Explain an error.** *"What does SIGNAGE_CONTENT_STALE mean and how do I fix it?"*
- [ ] **3.E3** **End of sale.** *"Which device models reach end-of-sale soonest?"*

Advanced

- [ ] **3.A1** **Open vs. resolved.** *"Summarize events across the organization, then check each device's history and tell me which issues are still open."*
- [ ] **3.A2** **Triage table.** *"Build a triage table: device, serial, room, open issue, severity and recommended action."*
- [ ] **3.A3** **Risk by usage.** *"Combine lifecycle dates with room popularity. Which replacement matters most?"*
- [ ] **3.A4** **Software currency.** *"Which devices are behind on RoomOS, and does that line up with any device issues?"*

Lab 4 — Configuration and drift

Goal: You'll learn to find configuration drift and decide what to change.

Time: Easy about 10 min · Advanced about 15 min.

Tools: `device_configuration_schema`, `device_configuration_template_list`, `device_configuration_list`, `device_configuration_diff`, `device_configuration_template_diff`.

Easy

- [ ] 4.E1 **What can be set.** *"List the configuration keys and their allowed values."*
- [ ] 4.E2 **Templates.** *"What configuration templates exist?"*
- [ ] 4.E3 **One device.** *"Show the Boardroom Room Bar Pro's settings."*

Advanced

- [ ] 4.A1 **Fleet drift report.** *"Check every device against its template and tabulate the drift."*
- [ ] 4.A2 **Rule out config.** *"Workshop Bay reported PROXIMITY_PAIRING_FAIL. Is its configuration the cause?"*
- [ ] 4.A3 **Same model, different rooms.** *"Diff the two Room Bars (Boardroom and Workshop)."*
  - Discuss: does a bigger room justify the louder default?
- [ ] 4.A4 **Remediation plan.** *"Write the exact changes needed to bring every device back to template, and validate each value against the schema."*
- [ ] 4.A5 **Question the template.** *"Lab Bench and Workshop Bay are open workspaces, but both use the meeting-room template. Is that the right template?"*

Lab 5 — Locations and the Firehose replay

Goal: You'll learn to read the raw event stream and reconcile what its counts say.

Time: Easy about 10 min · Advanced about 15 min.

Tools: `location_search`, `firehose_health`, `firehose_latest`, `firehose_events`.

Tip: `firehose_events` returns at most 50 events per pull. Before you count or trace anything, check whether the reply says `truncated: true`. If it does, ask the assistant to follow `nextOffset`, or to filter by `eventType`.

Easy

- [ ] 5.E1 **Site map.** *"Show me the location hierarchy."*
- [ ] 5.E2 **Stream health.** *"Is the Firehose healthy, and what does it subscribe to?"*
- [ ] 5.E3 **Who's on which floor.** *"Where are the tracked phones right now?"*

Advanced

- [ ] 5.A1 **Event mix.** *"Pull Firehose events and count them by type."*
- [ ] 5.A2 **Rebuild a meeting.** *"Using SPACE_OCCUPANCY_CHANGE events only, trace Boardroom North's headcount."*
- [ ] 5.A3 **Visitor journey.** *"Follow mockup-phone-01 across all event types in time order."*

- [ ] **5.A4** **Reconcile the counts.** *"North L1 reports DEVICE_COUNT, CAMERA_COUNT and USER_PRESENCE totals. Why don't they agree?"*
- [ ] **5.A5** **Spot personal data.** *"Which Firehose fields would be personal data in a real tenant?"*

Stretch goals

These have no single expected answer. Each one lists what a complete result includes. S1, S3 and S4 link to checklists and worked numbers in the Answer key.

- [ ] **S1** **Campus health report in one prompt.** Ask for a full report covering environment, utilization, device issues, drift and lifecycle.
- [ ] **S2** **Room profile.** *No answer key.* Pick one room and build a one-page profile by joining the tools on their shared IDs. A complete profile has 8 parts: location path, device, RoomOS, config drift, open issues, lifecycle dates, utilization and environment peak. If you pick Phone Booth N2, which has no device, say which parts don't apply.
- [ ] **S3** **Anomaly hunt.** Find at least three things in the data that don't add up, with the tool output that shows each one. Starting points:
  - Device telemetry and room metrics disagree on temperature.
  - Phone Booth N2 has environment readings but no device.
  - Both open workspaces, Lab Bench and Workshop Bay, use a meeting-room template.
  - Every device's telemetry reports exactly 22.8 °C and 50% humidity, whatever the headcount.
  - Device telemetry and room metrics disagree on headcount in one room.
  - An IoT sensor on South L1 (`mockup-sensor-lab-01`) reports a different temperature from Lab Bench's metrics for the same hour. Its event names only the floor, not a room, so tying it to Lab Bench rests on the sensor's name.
  - A phone's latest position comes after its exit event.
- [ ] **S4** **Predict and flag.** Use your Lab 1 model to predict Boardroom North's conditions with 15 people. Then propose a headcount limit that keeps the room under 24 °C.
- [ ] **S5** **Repeatable report.** *No answer key.* Write one prompt that produces the same weekly report every time. Success: run it twice in fresh sessions and get the same numbers and the same structure. Timestamps will differ, because the sample data moves with the clock.
- [ ] **S6** **Runbook per error code.** *No answer key.* For each of the three error codes, write the tool sequence an on-call engineer should follow, from detection to fix. Success: each runbook names the tools in order and ends with the catalog's recommended action.
- [ ] **S7** **Design the next tool.** *No answer key.* Name one question you couldn't answer in these labs, such as who booked a room or what the HVAC set point is. Specify the tool that would answer it: name, inputs, output fields, and which existing IDs it joins on.

Appendix: client setup

Set up one client, then go back to Confirm it works.

Claude Code CLI (Linux / macOS)

1. **Create a lab folder.** Work in a new folder so the lab files stay together. It's safe to run this again if the folder already exists:

   ```
   mkdir -p spaces-lab && cd spaces-lab
   ```

2. **Set your token.** This sets it for this terminal only. If you open a new terminal later, run it again there:

   ```
   export MCP_BEARER_TOKEN='Found-In-Box-Folder'
   ```

3. **Add the MCP server.** This command adds `spaces-mockup` to `.mcp.json` in the lab folder. It creates the file if it doesn't exist. If the file already exists, it adds the new entry and keeps your other servers. Keep the single quotes around the header: they stop your shell from filling in `${MCP_BEARER_TOKEN}`, so the real token is never written to the file. Claude fills it in from your environment when it starts. Paste all three lines together; they're one command:

   ```
   claude mcp add --transport http --scope project spaces-mockup \
     https://spaces-mockup.mcp.buildermate.ai/mcp \
     --header 'Authorization: Bearer ${MCP_BEARER_TOKEN}'
   ```

   If it says `MCP server spaces-mockup already exists in .mcp.json`, you've added it before and can go on to the next step. To replace the old entry instead, run `claude mcp remove spaces-mockup -s project`, then run the command again.

4. **Launch Claude** from the same shell. Claude may ask two questions: first whether to trust this folder (say yes), then whether to use the project MCP servers in `.mcp.json` (approve `spaces-mockup`):

   ```
   claude
   ```

5. **Verify.** In Claude, run `/mcp` and check that `spaces-mockup` shows as connected. If it doesn't:
   - **401 error or `Missing environment variables: MCP_BEARER_TOKEN`:** the token isn't set in the shell Claude was launched from. Exit Claude, run the `export` from step 2 in this shell, and relaunch.
   - **Scope shows "User config" instead of "Project config":** you have an older `spaces-mockup` entry that takes priority over `.mcp.json`. Exit Claude, run `claude mcp remove spaces-mockup -s user`, and relaunch.

6. **Coming back later.** You don't need to add the server again. In a new terminal, go to the lab folder, set the token, and launch Claude:

```
cd spaces-lab
export MCP_BEARER_TOKEN='Found-In-Box-Folder'
claude
```

Claude Code CLI (Windows)

Use PowerShell for every step, not Command Prompt. The prompt should start with `PS`, for example `PS C:\Users\you>`. Run every step in the same PowerShell window.

1. **Create a lab folder.** It's safe to run this again if the folder already exists:

```
New-Item -ItemType Directory -Force spaces-lab | Out-Null
cd spaces-lab
```

2. **Set your token.** This sets it for this PowerShell window only. If you open a new window later, run it again there:

```
$env:MCP_BEARER_TOKEN = "Found-In-Box-Folder"
```

3. **Add the MCP server.** This command adds `spaces-mockup` to `.mcp.json` in the lab folder. It creates the file if it doesn't exist. If the file already exists, it adds the new entry and keeps your other servers. Keep the single quotes around the header: they stop PowerShell from filling in `${MCP_BEARER_TOKEN}`, so the real token is never written to the file. Claude fills it in from your environment when it starts. Paste all three lines together; they're one command:

```
claude mcp add --transport http --scope project spaces-mockup `
  https://spaces-mockup.mcp.buildermate.ai/mcp `
  --header 'Authorization: Bearer ${MCP_BEARER_TOKEN}'
```

   If it says `MCP server spaces-mockup already exists in .mcp.json`, you've added it before and can go on to the next step. To replace the old entry instead, run `claude mcp remove spaces-mockup -s project`, then run the command again.

4. **Launch Claude** from the same PowerShell window. Claude may ask two questions: first whether to trust this folder (say yes), then whether to use the project MCP servers in `.mcp.json` (approve `spaces-mockup`):

```
claude
```

5. **Verify.** In Claude, run `/mcp` and check that `spaces-mockup` shows as connected. If it doesn't:

- **401 error or `Missing environment variables: MCP_BEARER_TOKEN`:** the token isn't set in the window Claude was launched from. Exit Claude, run the `$env:` command from step 2 in this window, and relaunch.
- **Scope shows "User config" instead of "Project config":** you have an older `spaces-mockup` entry that takes priority over `.mcp.json`. Exit Claude, run `claude mcp remove spaces-mockup -s user`, and relaunch.
- **`running scripts is disabled on this system` when you run `claude`:** PowerShell is blocking the script that npm installed for Claude Code. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer `Y`, then run `claude` again.

6. **Coming back later.** You don't need to add the server again. In a new PowerShell window, go to the lab folder, set the token, and launch Claude:

```
cd spaces-lab
$env:MCP_BEARER_TOKEN = "Found-In-Box-Folder"
claude
```

Claude Desktop (Windows / macOS)

Claude Desktop connects through a small extension, `spaces-mockup.mcpb`. It runs on the Node.js built into Claude Desktop, so you don't need to install anything else, and the token is stored in your system's credential store rather than in a config file.

1. **Download the extension.** Get `spaces-mockup.mcpb` from the lab's Box folder.
2. **Install it.** Double-click `spaces-mockup.mcpb`, or drag it onto the Claude Desktop window. You can also go to **Settings → Extensions → Advanced settings → Install Extension…** and choose the file. Click **Install** when Claude Desktop shows the extension details. If double-clicking opens nothing, or Windows asks **How do you want to open this file?**, close that prompt and use the drag-and-drop or **Install Extension…** route instead. Claude Desktop must be open for either.
3. **Enter your token.** Do this even if the install window didn't ask for it. Installing doesn't always prompt for the token, and the extension can't connect without it.
    1. Go to **Settings → Extensions** and click **Spaces Mockup (Lab)**.
    2. Click **Configure**.
    3. In **Bearer token**, paste only the token from the `.txt` file. Don't type `Bearer` in front of it, and don't add quotes.
    4. Click **Save**.
4. **Restart the extension.** The extension reads the token only when it starts, so a token saved while it's running isn't used until it restarts. In **Settings → Extensions**, turn **Spaces Mockup (Lab)** off and then on again. If it was already off, just turn it on.
5. **Verify.** Start a new chat and send *"Call firehose_health."* Allow the tool when Claude Desktop asks. The reply should say the status is ok. If you get an error instead:
    - **"No bearer token is set for spaces-mockup":** Claude Desktop started the extension without a token. Do steps 3 and 4 again. Make sure you click **Save** before you turn the extension off and on.

- **"rejected the bearer token (HTTP 401)":** the extension sent a token, but it's not the right one. Copy it again from the `.txt` file, paste it into **Configure**, save, and turn the extension off and on.
- **Tools don't appear:** quit Claude Desktop fully and reopen it. On Windows, right-click the Claude icon in the system tray and choose **Quit**. On macOS, press **Cmd+Q**. Closing the window isn't enough.
- **Still stuck:** the extension's log is in **Settings → Developer**. Lines starting with `[spaces-mockup]` say whether the token was missing or rejected.

Cursor

Cursor connects to the server directly and reads the token from an environment variable, so it stays out of the config file.

1. **Set your token.** Cursor only sees the variable if it's set before Cursor starts, so quit Cursor completely first.
   - **Windows (PowerShell):** run this to save the token for your user account, then reopen Cursor:

     ```
     setx MCP_BEARER_TOKEN "Found-In-Box-Folder"
     ```

   - **Linux / macOS:** export it, then open Cursor from the same shell. Opening Cursor from the Dock or a launcher doesn't pass the variable on. If `cursor` isn't found, open the Command Palette in Cursor and run **Shell Command: Install 'cursor' command**, then try again:

     ```
     export MCP_BEARER_TOKEN='Found-In-Box-Folder'
     cursor .
     ```

2. **Add the MCP server.** Open your global Cursor MCP file in the editor: `~/.cursor/mcp.json` on Linux and macOS, or `%USERPROFILE%\.cursor\mcp.json` on Windows. Create the file if it doesn't exist. You can also reach it from Cursor's MCP settings; the menu names change between Cursor versions. Add the following. If the file already has an `mcpServers` block, add only the `spaces-mockup` entry inside it, so your existing servers are kept. Save the file:

```
{
  "mcpServers": {
    "spaces-mockup": {
      "url": "https://spaces-mockup.mcp.buildermate.ai/mcp",
      "headers": {
        "Authorization": "Bearer ${env:MCP_BEARER_TOKEN}"
      }
    }
  }
}
```

Cursor uses `${env:...}`, not the `${...}` form that Claude Code uses.

3. **Verify.** In Cursor's MCP settings, `spaces-mockup` should be enabled and list its tools. If it shows an error or no tools:
   - **401 or "bearer required":** the variable wasn't set when Cursor started. On Windows, quit Cursor fully and reopen it. On Linux or macOS, quit Cursor and relaunch it from the shell where you ran `export`.
   - **Anything else:** open the Output panel (**Ctrl+Shift+U**, or **Cmd+Shift+U** on macOS) and choose **MCP Logs** to see the error.
4. **Use it in Agent mode.** Open the chat panel, switch to **Agent**, and ask the lab questions there. Cursor asks before each tool call unless you allow it to run them automatically.

Codex CLI

Codex reads the server from its config file and the token from an environment variable, so the token stays out of the file.

1. **Set your token.** This sets it for this terminal only, so launch Codex from the same terminal. If you open a new terminal later, run it again there:
   - **Linux / macOS:**

     ```
     export MCP_BEARER_TOKEN='Found-In-Box-Folder'
     ```

   - **Windows (PowerShell):**

     ```
     $env:MCP_BEARER_TOKEN = "Found-In-Box-Folder"
     ```

2. **Add the MCP server.** Open your Codex config file in an editor: `~/.codex/config.toml` on Linux and macOS, or `%USERPROFILE%\.codex\config.toml` on Windows. Create the file if it doesn't exist. Add the following at the end and save it. Keep any other `[mcp_servers.…]` sections already in the file; if it already has a `[mcp_servers.spaces-mockup]` section, replace that one instead of adding a second:

```
[mcp_servers.spaces-mockup]
url = "https://spaces-mockup.mcp.buildermate.ai/mcp"
bearer_token_env_var = "MCP_BEARER_TOKEN"
```

`bearer_token_env_var` takes the variable's name, not the token. Codex reads the token from your environment and sends the `Authorization: Bearer` header itself.

3. **Launch Codex** from the same terminal:

```
codex
```

4. **Verify.** In Codex, run `/mcp` and check that `spaces-mockup` is listed with its tools. If it isn't:
   - **401 or no tools:** the token isn't set in the terminal Codex was launched from. Exit Codex, run the command from step 1 in this terminal, and relaunch.
   - **Codex reports a config error on startup:** check the section in `config.toml` is copied exactly, including the quotes, and that `[mcp_servers.spaces-mockup]` appears only once.
5. **Coming back later.** You don't need to add the server again. In a new terminal, set the token as in step 1 and run `codex`.

Codex desktop app

The Codex desktop app (now part of the ChatGPT desktop app) uses the same `config.toml` as Codex CLI. If you've already set up Codex CLI on this computer, step 2 is done. The app's **MCP servers** settings screen has no field for a bearer token, so you add the server in the file and set the token where the app can read it.

1. **Set your token.** An app opened from the Dock, Start menu or a launcher doesn't see a token you `export` in a terminal, so set it like this, then quit the app completely:
   - **Windows (PowerShell):** run this to save the token for your user account:

     ```
     setx MCP_BEARER_TOKEN "Found-In-Box-Folder"
     ```

   - **macOS:** run this in Terminal. It lasts until you restart your Mac:

     ```
     launchctl setenv MCP_BEARER_TOKEN 'Found-In-Box-Folder'
     ```

   - **Linux:** export it, then start the app from the same terminal with `chatgpt`:

     ```
     export MCP_BEARER_TOKEN='Found-In-Box-Folder'
     chatgpt
     ```

2. **Add the MCP server.** Open `~/.codex/config.toml` on Linux and macOS, or `%USERPROFILE%\.codex\config.toml` on Windows, and add the section from Codex CLI step 2. Keep any other `[mcp_servers.…]` sections, and have only one `[mcp_servers.spaces-mockup]` section.
3. **Open the app and restart the server.** Reopen the app. On Linux, use the terminal from step 1. Go to **Settings → MCP servers**, find `spaces-mockup`, and select **Restart**.
4. **Verify.** In a new chat, type `/mcp` and check that `spaces-mockup` is listed as connected. If it isn't:
    - **401 or no tools:** the app started without the token. Do step 1 again, quit the app completely, and reopen it. On Windows, `setx` only applies to apps started after you run it. On macOS, run `launchctl setenv` again if you've restarted your Mac since.
    - **`spaces-mockup` isn't in the list:** check the section in `config.toml` is copied exactly, then restart the app.
    - **Windows with Codex running in WSL:** the app reads `%USERPROFILE%\.codex\config.toml` on Windows, not the config in your Linux home folder. Add the section to the Windows file.

After the lab (optional)

The token only works for this lab, but you can remove it and the server entry when you're done:

- **Claude Code:** delete the `spaces-lab` folder. The token was only set for that terminal or window, so it's gone once you close it.
- **Claude Desktop:** in **Settings → Extensions**, open **Spaces Mockup (Lab)** and uninstall it. This also removes the saved token.
- **Cursor:** remove the `spaces-mockup` entry from `mcp.json`. On Windows, also remove the saved token in PowerShell:

    ```
    [Environment]::SetEnvironmentVariable("MCP_BEARER_TOKEN", $null,
    "User")
    ```

- **Codex CLI:** remove the `[mcp_servers.spaces-mockup]` section from `config.toml`. The token was only set for that terminal or window, so it's gone once you close it.
- **Codex desktop app:** remove the `[mcp_servers.spaces-mockup]` section from `config.toml`. It's the same file Codex CLI uses, so this removes the server from both. Then remove the saved token. On Windows, run this in PowerShell:

    ```
    [Environment]::SetEnvironmentVariable("MCP_BEARER_TOKEN", $null,
    "User")
    ```

    On macOS, run `launchctl unsetenv MCP_BEARER_TOKEN`, or restart your Mac.

Answer key

Numbers like **2.A3** mean Lab 2, Advanced track, exercise 3. **S** numbers are stretch goals.

Lab 1 — Rooms and environment

↩ Back to Lab 1

1.E1 Temperature everywhere

- Every room's latest reading is 22.4–22.85 °C. Workshop Bay is the warmest (22.85 °C, 3 people). Earlier readings run higher; see 1.E2.
- "Right now" means each room's latest reading, which is 1 to 3 hours old depending on the room; see 1.A4.

1.E2 One room over the day

- It starts at 22.4 °C, peaks at 24.2 °C and ends back at 22.4 °C.

1.E3 Quietest rooms

- Focus Desk 12 and Phone Booth N2. Their noise never goes above 30.

1.A1 Comfort alerts

- One hour only. Boardroom North hit 24.2 °C and 60% with 12 people present.

1.A2 Build a model

- The same rule fits every room. Temperature is 22.4 °C + 0.15 °C per person, humidity is 48% + 1% per person, and noise is 28 + 2 per person.
- To check it: 7 people in Workshop Bay predicts 23.45 °C, 55% and 42, which matches the data.
- Real rooms wouldn't behave like this: one extra person warms a phone booth far more than a boardroom. The rule is a property of the sample data, so S4's prediction only holds inside this dataset.

1.A3 Two sources, one room

- They disagree. The metrics say 24.2 °C at the 12-person peak, while device telemetry says 22.8 °C.
- Noise disagrees too: the metrics say 52 and the telemetry's `ambientNoise` says 38. Both sources agree on the headcount of 12.
- Discussion: good answers notice that the telemetry reads a flat 22.8 °C and 50% in every room, from 1 person to 12 (see S3), while the metrics rise with headcount. For a comfort complaint, trust the source that moves with conditions, here the room metrics, and treat the device sensor as suspect until someone checks it.

1.A4 Data freshness

- Boardroom North has 11 hourly points, Huddle A 9, Lab Bench 5, Focus Desk 12 and Workshop Bay 4 each, and Phone Booth N2 only 3.
- The latest readings are about 1 hour old for Boardroom North and Lab Bench, about 2 hours for Huddle A, Focus Desk 12 and Workshop Bay, and about 3 hours for Phone Booth N2.

Lab 2 — Utilization and occupancy

↩ Back to Lab 2

2.E1 Who's in now

- Workshop Bay 3, Huddle A 1, Lab Bench 1, all others 0. No room is booked right now.
- These are each room's latest reading, which can be up to 3 hours old; see 2.A5.

2.E2 Most popular

- Boardroom North 450 min, Huddle A 400, Lab Bench 250, Workshop Bay 200, then Focus Desk 12 and Phone Booth N2 at 50 each.

2.E3 Wrong-sized rooms

- Boardroom North is overcrowded (peak 12 people in 10 seats, ratio 1.2). Focus Desk 12 and Phone Booth N2 are under-utilized.
- They're flagged on minutes used (50 each, under the tool's 80-minute line), not on crowding. Each one's peak is 1 person in 1 seat, a ratio of 1.0.

2.A1 Booking accuracy

- Boardroom 1.07, Huddle A 0.83, Lab Bench 2.08, Workshop Bay 1.67. Focus Desk and Phone Booth are never booked.
- Read the ratio with care. Each used hour counts 50 minutes and each booked hour 60, so a room that is booked and used every hour scores 0.83, not 1. Above 0.83 means walk-ins; below it means no-shows.
- Huddle A scores exactly 0.83, but not because every booking was used: one walk-in hour and one no-show cancel out (see 2.A2).
- Lab Bench is the only room where most use is unbooked, 3 of its 5 hours. Workshop Bay is half, 2 of 4.

2.A2 No-shows and walk-ins, hour by hour

- One booked-but-empty hour: Huddle A, about 6 hours ago, between two runs of bookings. The metrics show `booked: true` with 0 people.
- Hours used without a booking: Lab Bench 3, Boardroom North 2, Workshop Bay 2, and 1 each for Huddle A, Focus Desk 12 and Phone Booth N2.

2.A3 Live vs. history

- Yes for headcount (for example Workshop Bay 3, Boardroom 0). Most timestamps agree within a few seconds; Boardroom North's snapshot is about 30 minutes newer than its last metrics point.

2.A4 Make the case

- A strong answer uses the capacity tool (Boardroom at 1.2×) and the usage tool (Focus Desk and Phone Booth at 50 min, never booked).
- No room can take the 12-person stand-up: Boardroom North is the biggest room, at 10 seats. Good recommendations split the stand-up or cap it at 10 (the S4 limit), and might repurpose Focus Desk 12 or Phone Booth N2.

2.A5 Trust the snapshot

- Each room's entry has its own age: Boardroom North about 30 minutes, Lab Bench about 1 hour, Huddle A, Focus Desk 12 and Workshop Bay about 2 hours, and Phone Booth N2 about 3 hours.
- Only Boardroom North's is recent enough to act on. Phone Booth N2 being empty 3 hours ago says little about now, and Huddle A's 1 person is 2 hours old. A good answer says how old a reading may be before you stop trusting it.

Lab 3 — Devices, errors and lifecycle

↩ Back to Lab 3

3.E1 Fleet status

- 5 devices. Four are connected, and the Lab Bench Board Pro 55 shows warning.

3.E2 Explain an error

- The signage playlist hasn't refreshed in 24 hours. Push the workspace playlist from Control Hub.

3.E3 End of sale

- The Room Kit Mini already reached end-of-sale on 31 Oct 2026. Next is the Desk Pro on 30 Jun 2027.

3.A1 Open vs. resolved

- 5 events (1 error, 2 warnings, 2 info). The `"none"` code in the totals is Boardroom North's `call.started` info event, not an error.
- `CAM_HDMI_SIGNAL_LOST` counts 2 in the totals, but only one is an error. The other is Huddle A's info-level `camera.recovered` event, which carries the same code.
- Huddle A's `CAM_HDMI_SIGNAL_LOST` is resolved, because a recovery event followed about an hour later.
- Still open: Lab Bench `SIGNAGE_CONTENT_STALE` and Workshop Bay `PROXIMITY_PAIRING_FAIL`.

3.A2 Triage table

- 2 rows. `MOCKBRD004` at Lab Bench (warning, push the signage playlist), and `MOCKBAR005` at Workshop Bay (warning, confirm `Proximity.Mode` is On and that ultrasound isn't blocked).

3.A3 Risk by usage

- The Room Kit Mini. It passed end-of-sale on 31 Oct 2026 and sits in Huddle A, the second-most-used room (400 min). Software maintenance for it ends 31 Oct 2029.

3.A4 Software currency

- The Board Pro 55 (11.26.1.1) and Room Kit Mini (11.28.1.3) trail 11.32.1.5.
- Both have had events, and the Board Pro 55 is the only device in warning. That's a correlation worth checking, not proof of a cause.

Lab 4 — Configuration and drift

↩ Back to Lab 4

4.E1 What can be set

- 5 keys. `Audio.DefaultVolume` takes 0–100, `Video.Selfview.Default` takes Off / Current / PIP, and `Bookings.Enabled`, `Standby.Control` and `Proximity.Mode` are on/off or true/false. All 5 apply to every model.

4.E2 Templates

- Meeting Room Standard and Personal Desk Standard. Both control the same 5 keys.

4.E3 One device

- Volume 50, self-view Off, bookings True, standby On, proximity On. Its template is Meeting Room Standard.

4.A1 Fleet drift report

Device	Template	Drift
Lab Bench Board Pro 55	Meeting Room Standard	Volume 45 (template 40); Standby Off (template On)
Boardroom North Room Bar Pro	Meeting Room Standard	Volume 50 (template 40)
Huddle A Room Kit Mini	Meeting Room Standard	none
Workshop Bay Room Bar	Meeting Room Standard	none
Focus Desk 12 Desk Pro	Personal Desk Standard	none

4.A2 Rule out config

- No. `Proximity.Mode` is On and the device matches its template, so check the room for something blocking ultrasound.

4.A3 Same model, different rooms

- One key, `Audio.DefaultVolume` (50 vs. 40).
- Discussion: the size argument is weak. Workshop Bay seats 8 to Boardroom North's 10, yet it works at the template's 40. Good answers either bring Boardroom back to 40 (see 4.A4) or propose changing the template for every room, rather than leaving one device's drift in place.

4.A4 Remediation plan

- Lab Bench needs `Audio.DefaultVolume` 40 and `Standby.Control` On. Boardroom needs `Audio.DefaultVolume` 40. All values are valid under the schema.
- The server is read-only, so this is a plan, not a change.
- Check 4.A5 before changing Lab Bench's `Standby.Control`: the Off setting may be deliberate.

4.A5 Question the template

- There's no single right answer. Workshop Bay matches the template exactly, so nothing flags it. Lab Bench has drifted to Standby Off, likely on purpose, to keep the board's display on.
- Good answers weigh that drift against keeping to the standard, and ask whether open workspaces need a template of their own.

Lab 5 — Locations and the Firehose replay

↩ Back to Lab 5

5.E1 Site map

- Mockup Campus contains North Building (L1, L2) and South Building (L1).

5.E2 Stream health

- Status ok, pull channel, replay on, 14 event types. It is a sample replay, not the live Cisco Spaces Firehose.

5.E3 Who's on which floor

- 3 phones, all on North L1, with x/y map positions.
- The positions aren't equally fresh. `mockup-phone-03`'s is 8 hours older than `mockup-phone-02`'s, so treat it as where the phone was, not where it is.
- Fresh isn't the same as present. `mockup-phone-02` has the newest position, but a `DEVICE_PRESENCE` exit event shows it left North L1 about 5 minutes before that position was recorded. Check `firehose_events` for exits before saying a phone is still on the floor.

5.A1 Event mix

- The first pull returns 50 of 77 events, with `truncated: true` and `nextOffset: 50`. `SPACE_OCCUPANCY` is the most common type, at 22 of the 50.
- `IOT_TELEMETRY` and `KEEP_ALIVE` don't appear in the first pull even though they're subscribed. They're in the second page (`offset: 50`), or filter `firehose_events` by `eventType` to find 1 and 3 of them.
- A complete count covers all 77. If the assistant counted only 50, ask it to follow `nextOffset`.
- The full count is `SPACE_OCCUPANCY` 36, `SPACE_OCCUPANCY_CHANGE` 11, `DEVICE_PRESENCE` 6, `DEVICE_LOCATION_UPDATE` 6, `WEBEX_TELEMETRY` 5, `USER_PRESENCE` 3, `KEEP_ALIVE` 3, and 1 each of the other seven types.

5.A2 Rebuild a meeting

- 0 → 2 → 7 → 12 → 8 → 1 → 5 → 9 → 3 → 1 → 0. This matches the 12-person peak from Labs 1 and 2.
- If the answer stops at 5, the assistant used one unfiltered pull, which only holds the first 7 changes, and didn't follow `nextOffset`. Ask it to filter `firehose_events` by `eventType`.

5.A3 Visitor journey

- Device entry, user entry (`mockup-user-01`), Wi-Fi association, position (20, 30), inactive, position (21, 31), active, then position (21.5, 30.5).

5.A4 Reconcile the counts

- 18 associated plus 5 probing devices, a camera count of 14, and 12 users (8 identified, 4 anonymous).
- `DEVICE_PRESENCE` adds a fourth count: 12 active plus 3 inactive devices, 15 in all, which is still short of the 18 associated.
- The last `DEVICE_PRESENCE` event, `mockup-phone-02`'s exit, drops to 9 active plus 3 inactive. If the assistant quotes 9, it read the latest event rather than the one at the time of the other counts.
- A `RAW_CAMERA_COUNT` of 6 may also turn up. It's one camera zone (`zone-lobby`), not the whole floor, so don't compare it with the floor's camera count of 14.
- A good answer explains that people carry several devices, cameras miss some people, and identified users depend on sign-in.

5.A5 Spot personal data

- Email, first and last name, user ID, MAC address, IPv4, and latitude/longitude.

Stretch goals

↩ Back to the stretch goals

S1 Campus health report

A complete report includes:

- [ ] Boardroom North overcrowded: 12 in 10 seats, 24.2 °C
- [ ] Focus Desk 12 and Phone Booth N2 under-used: 50 min, never booked
- [ ] Huddle A no-show: one booked hour with nobody there
- [ ] Lab Bench Board Pro: stale signage, oldest RoomOS, two drifted keys
- [ ] Workshop Bay proximity failure ruled out as a config problem
- [ ] Room Kit Mini already past end-of-sale (31 Oct 2026), in the second-busiest room

S3 Anomaly hunt

Any three of these, each with the tool output that shows it:

- [ ] Telemetry vs. metrics: Boardroom North's `WEBEX_TELEMETRY` says 22.8 °C and noise 38 at the 12-person peak; `workspace_metrics` says 24.2 °C and noise 52. Noise disagrees in every room, for example Workshop Bay 44 vs. 42 and Lab Bench 41 vs. 38, but Boardroom North's gap is by far the widest
- [ ] Phone Booth N2 has 3 environment readings in `workspace_metrics` but no device in `workspace_search` or `device_search`
- [ ] Lab Bench and Workshop Bay are both `openWorkspace` but use `tpl-meeting-standard`
- [ ] All 5 `WEBEX_TELEMETRY` events report 22.8 °C and 50% humidity, from 1 person to 12
- [ ] Huddle A's `WEBEX_TELEMETRY` counts 4 people while `workspace_metrics` says 3 for the same hour; the other four rooms' counts agree
- [ ] `mockup-sensor-lab-01` reports 23.4 °C while Lab Bench's metrics say 23.15 °C for the same hour, and its event is tagged only to South L1
- [ ] `mockup-phone-02`'s `DEVICE_PRESENCE` exit event is followed about 5 minutes later by a `DEVICE_LOCATION_UPDATE` on North L1, which `firehose_latest` shows as its current position

S4 Predict and flag

About 24.65 °C, 63% humidity and noise 58 with 15 people. To stay under 24 °C, keep it to 10 people or fewer, which is exactly its seat count.